

ISPW Project Report 2025/2026

BY FRANCESCO EVANGELISTI



Project Name: **Dose Guard**

Index

1 Software Requirements Specification	2
1.1 Introduction	2
1.1.1 Aim of the document	2
1.1.2 Overview of the defined system	2
1.1.3 HW e SW requirements	3
1.1.4 Related systems	3
1.2 User Stories	4
1.3 Functional Requirements	4
1.4 Use Cases	5
1.4.1 Overview diagram	5
1.4.2 Internal Steps	5
2 Storyboards	6
2.1 Login	6
2.2 Homepage	7
2.4 Medicine Section	9
3 Design	10
3.1 Class diagram	10
3.1.1 VOPC	10
3.1.2 Design-Level Diagram	10
3.2 Design patterns	12
3.3 Activity diagram	14

3.3.1 UC: Book Appointment.....	14
3.3.2 UC: Login.....	15
3.4 Sequence Diagram	16
3.5 State Diagram.....	17
4 Testing	17
5 GitHub Repository	18
6 SonarCloud.....	18
7 Presentation Video.....	18

1 Software Requirements Specification

1.1 Introduction

1.1.1 Aim of the document

The purpose of this document is to present the fundamental and specific requirements of the “Dose Guard” system, a project for the Software Engineering and Web Design course, Academic Year 2025/2026. The document provides a comprehensive overview of the system’s objectives, core functionalities, and design choices to guide the development process.

1.1.2 Overview of the defined system

The system is a Java-based software platform structured according to the MVC/MVP architecture, developed using Maven, and featuring graphical user interfaces implemented through JavaFX and a command-line interface.

The system supports the efficient and modern management of medical appointments and medication tracking, including the following functionalities:

- Management of patients, doctors, and pharmacists.
- Booking and management of medical appointments based on medical specialization and doctor availability.
- Visualization of doctor availability and patient therapy schedules.
- Creation and management of prescriptions and therapy assignments between doctors and patients.
- Monitoring patient progress, adherence rates, and completion of therapeutic plans.

- Email notifications for booking confirmations, cancellations, and prescription updates.
- Viewing reports and statistical data to support clinical monitoring and decision-making processes.

There are three user types with different functionalities: Patients, Doctors, and Pharmacists.

1.1.3 HW e SW requirements

Software requirements:

- Java 17+
- Maven 3.8+
- Scene Builder (for generating FXML files)
- Draw.io (for creating UML diagrams)
- IDE: IntelliJ IDEA or Eclipse
- MySQL Server 8.0+

Hardware requirements:

- CPU: Dual-Core 2.0 GHz
- RAM: 2 GB
- Disk Space 1 GB
- Operating System: Windows, MacOS, Linux (compatible with java 17+)

1.1.4 Related systems

DoseGuard:

- **Pros:** Integrated multi-role ecosystem managing patients, doctors, and pharmacists, direct clinical prescription handling, structured time slot booking based on medical specializations, adherence tracking, and complete architectural customization.
- **Cons:** Lack of a native mobile application (iOS/Android), closed ecosystem without third-party wearable integrations, and limited push notification capabilities compared to commercial apps.

MediSafe

- **Pros:** Cross-platform mobile availability (iOS/Android) with aggressive push notifications, "Medfriend" caregiver alert features, built-in drug interaction database, and extensive health/vital sign trackers.

- **Cons:** Freemium business model requiring paid subscriptions for advanced features, decoupling from active real-time medical prescriptions, and privacy considerations regarding cloud-based data storage.

Mediately

- **Pros:** Extensive and frequently updated clinical drug database, specialized advanced drug-drug interaction checker, multi-platform support, and robust offline capabilities for healthcare professionals.
- **Cons:** Not a full practice management suite (lacks end-to-end appointment scheduling or booking slots), limited patient-side engagement for personal therapy management, and tiered professional subscription models.

1.2 User Stories

1. As a patient, I want to book an appointment with a doctor by specialization and availability, so that I can easily find and secure a visit without wasting time.
2. As a doctor, I want to schedule an appointment for a patient, so that I can ensure they receive automatic reminders.
3. As a pharmacist, I want to verify and confirm a digital prescription received from a doctor, so that I can safely dispense the correct medication.

1.3 Functional Requirements

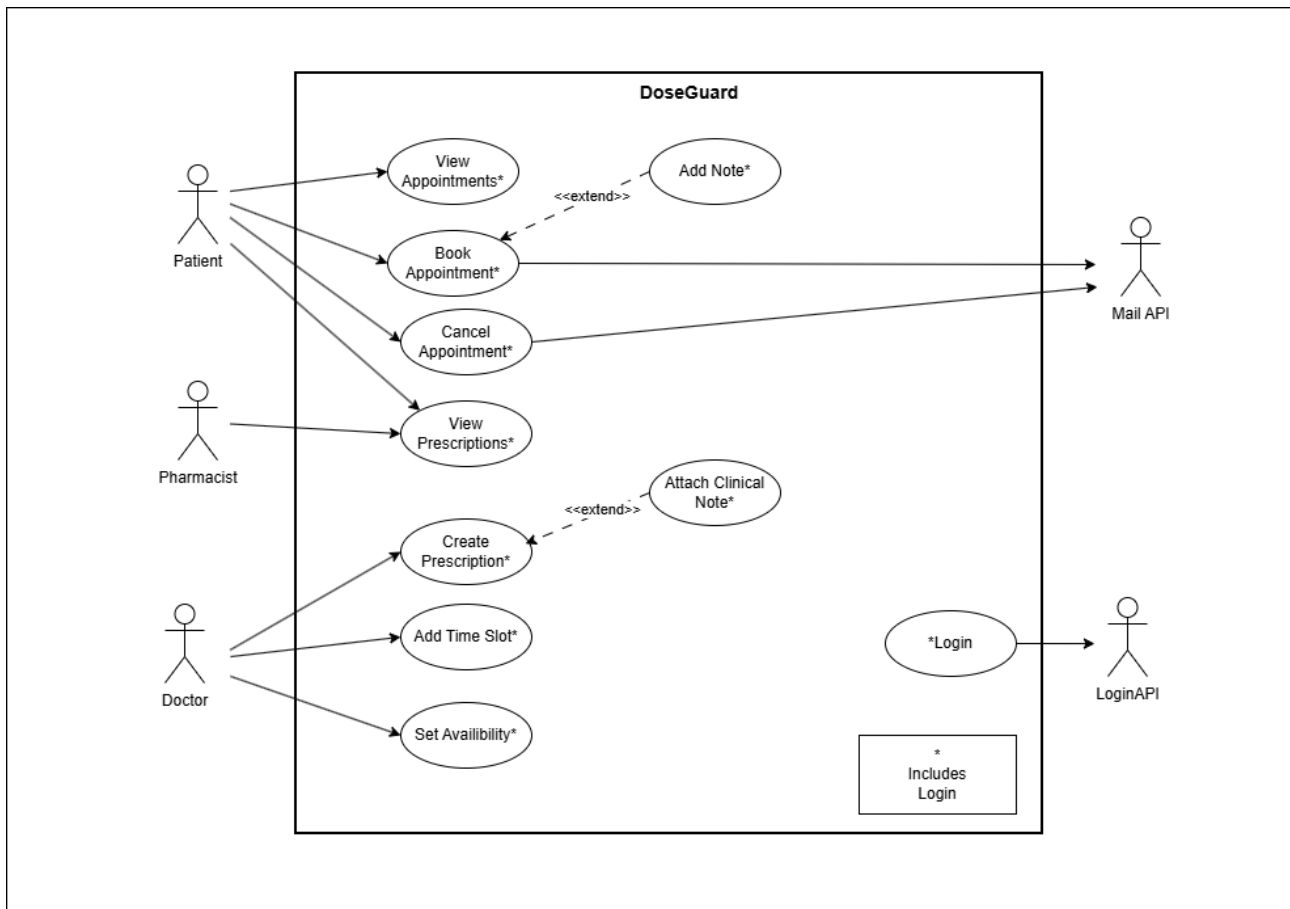
1. The system shall send reminders to the patient for upcoming appointments.
2. The system shall display to the patient the available time slots matching the doctor and date selected.
3. The system shall record the prescription entered by the doctor and associate it with the patient's profile.

Glossary

Slot = A predefined time interval created by a doctor during which they are available for a visit. Once booked, the slot is marked as unavailable.

1.4 Use Cases

1.4.1 Overview diagram



1.4.2 Internal Steps

Use case: Book Appointment

Main flow:

- 1: The patient accesses their account.
- 2: The patient selects the New Appointment function from the menu.
- 3: The system displays the list of medical specializations.
- 4: The patient selects a medical specialization.
- 5: The system displays the list of doctors associated with the selected specialization.
- 6: The patient selects a doctor.
- 7: The system displays the available time slots for the selected doctor.
- 8: The patient selects an available date and time slot.

9: The system verifies the availability of the selected time slot and registers the booking.

10: The system updates the time slot status to unavailable.

11: The system synchronizes the event with Google Calendar and sends a confirmation message to the Patient.

Extensions:

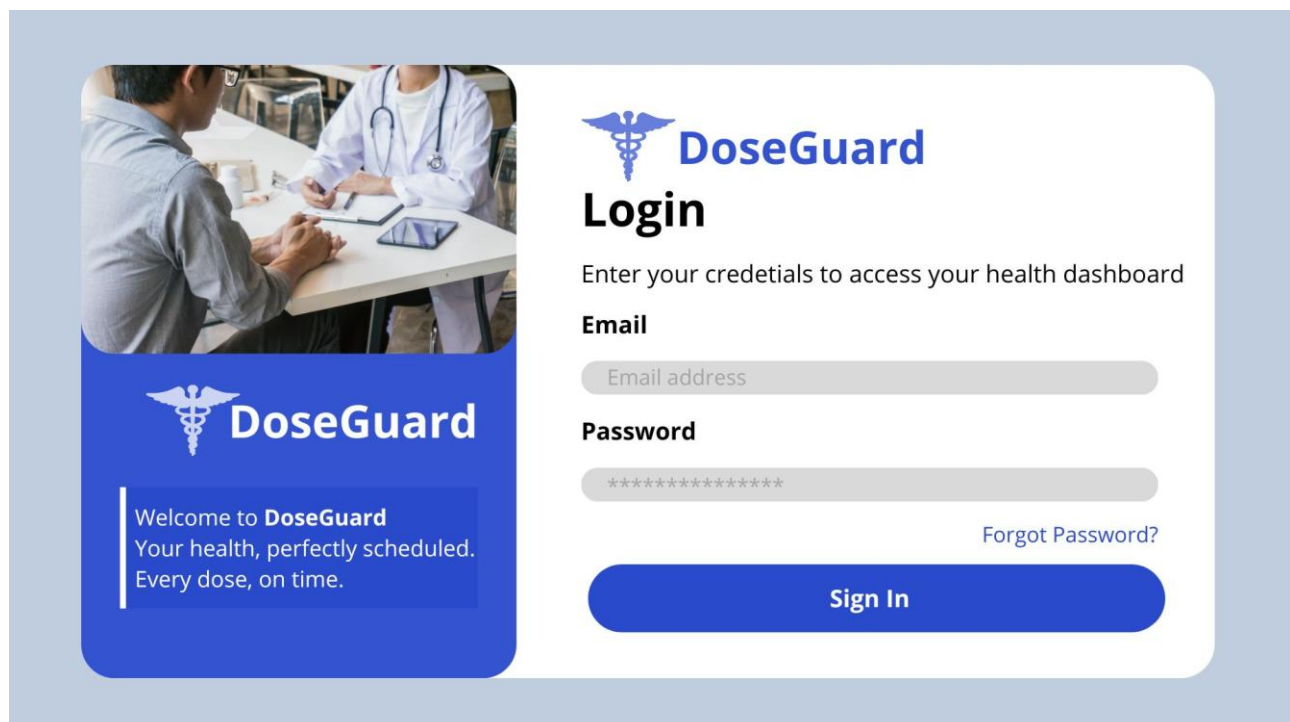
5a. No doctors available for the selected specialization: The system notifies the patient and returns them to the specialization selection screen (step 3).

7a. No available time slots for the selected doctor: The system notifies the patient and returns them to the doctor selection screen (step 5).

9a. The selected slot is no longer available: The system notifies the patient and returns them to the slot selection screen (step 7).

2 Storyboards

2.1 Login



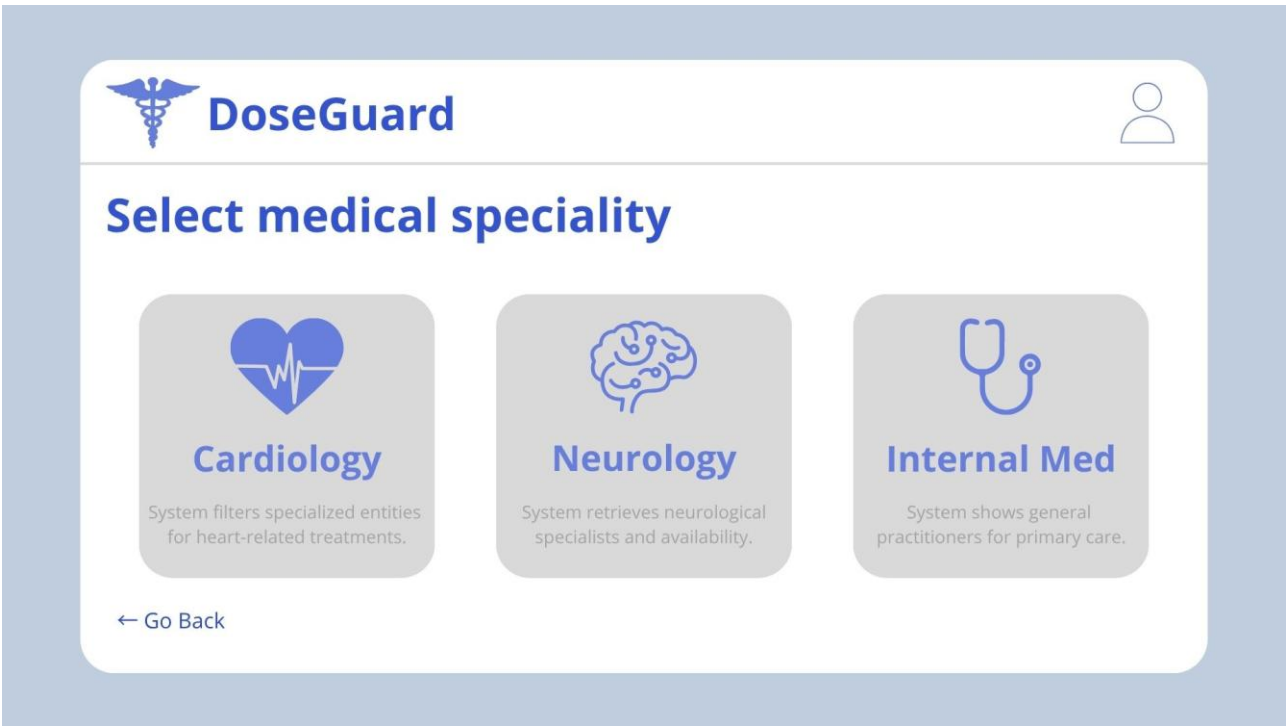
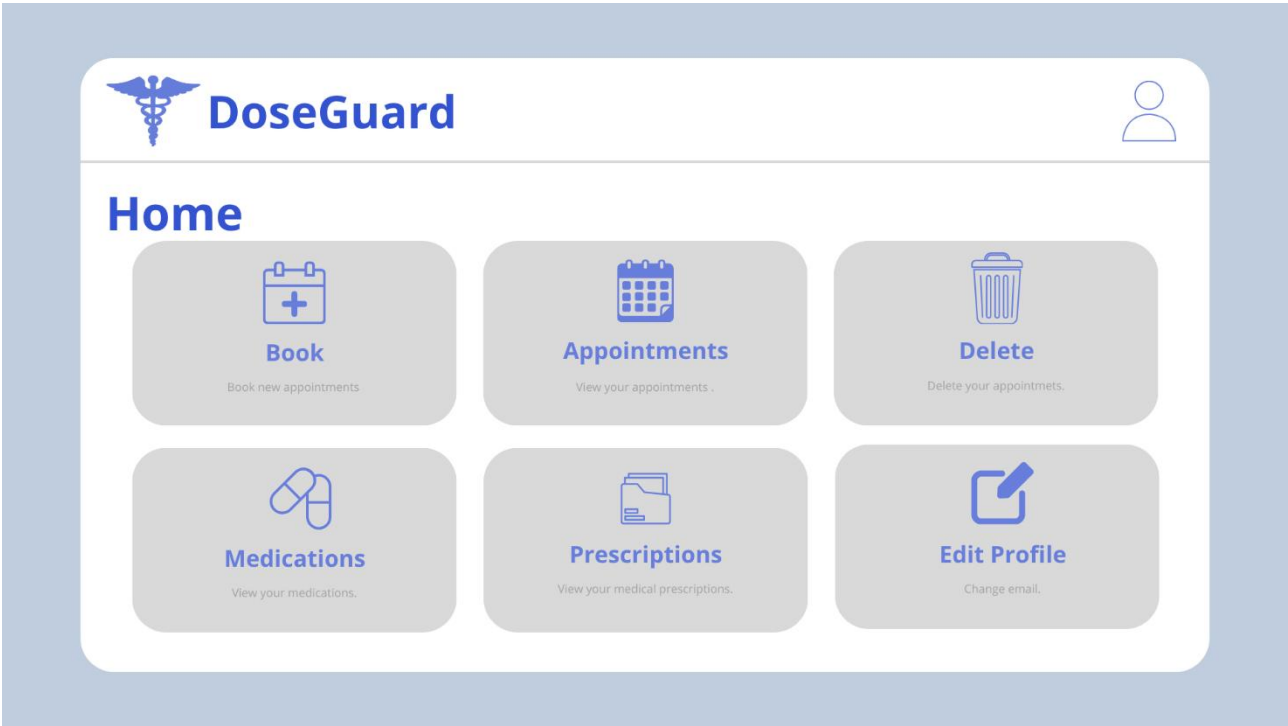
The image shows a login screen for the DoseGuard application. On the left, there is a vertical banner with a photo of a doctor and a patient at a desk. The banner features the DoseGuard logo and the text: "Welcome to DoseGuard", "Your health, perfectly scheduled.", and "Every dose, on time." The main login area on the right has a white background with rounded corners. It includes the DoseGuard logo and the word "Login" in large, bold, blue text. Below this, it says "Enter your credentials to access your health dashboard". There are two input fields: "Email" with a placeholder "Email address" and "Password" with a placeholder of ten asterisks. A "Forgot Password?" link is located to the right of the password field. At the bottom, there is a large blue button labeled "Sign In".

**DoseGuard**
Welcome to **DoseGuard**
Your health, perfectly scheduled.
Every dose, on time.

**DoseGuard**
Login
Enter your credentials to access your health dashboard
Email
Email address
Password

[Forgot Password?](#)
Sign In

2.2 Homepage





DoseGuard



Select a doctor



Dr. Smith

Cardiologist | 8 years exp.
Tor Vergata Hospital



Dr. Jones

Cardiologist | 20 years exp.
San Giovanni Hospital



Dr. Williams

Cardiologist | 35 years exp.
Umberto 1 Hospital

[← Go Back](#)



DoseGuard



Select a time slot



22/09/2026

Ore 15:30



01/10/2026

Ore 10:00



02/10/2026

Ore 09:30

[← Go Back](#)



DoseGuard



My Appointments

Confirmed (1)

Deleted (0)

Expired (1)

Cardiology

Doctor: Dr. Andrea Neri

Date: 01/10/2026 Time: 10:00

Email: dr.neri@test.com

[← Go Back](#)

2.4 Medicine Section



DoseGuard



My medications

To be hired

Tachipirina

Dosage: 1000 mg

Mark as taken

Hired

Augementin

Dosage: 1 gr

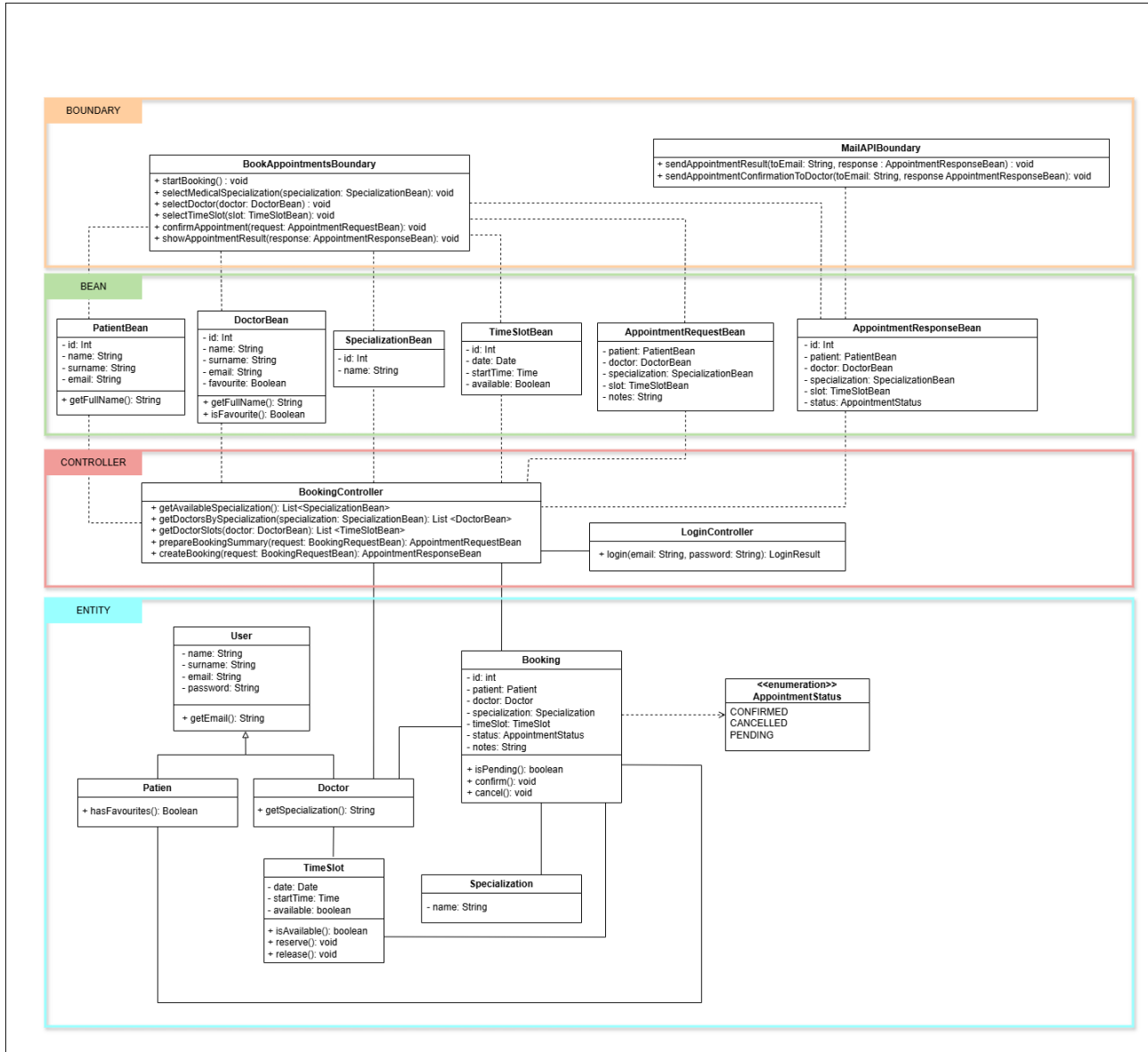
✓ Taken

[← Go Back](#)

3 Design

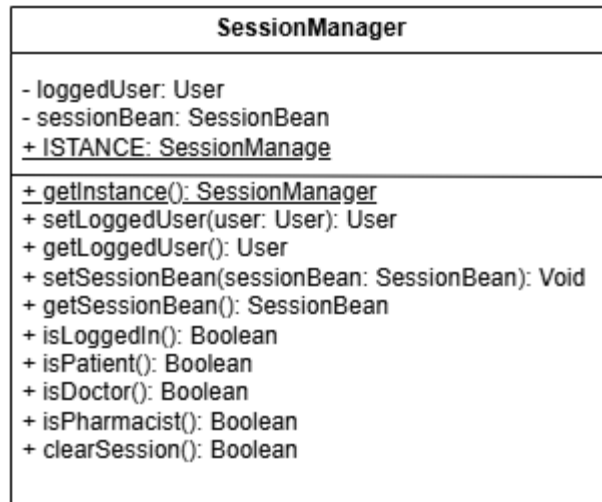
3.1 Class diagram

3.1.1 VOPC

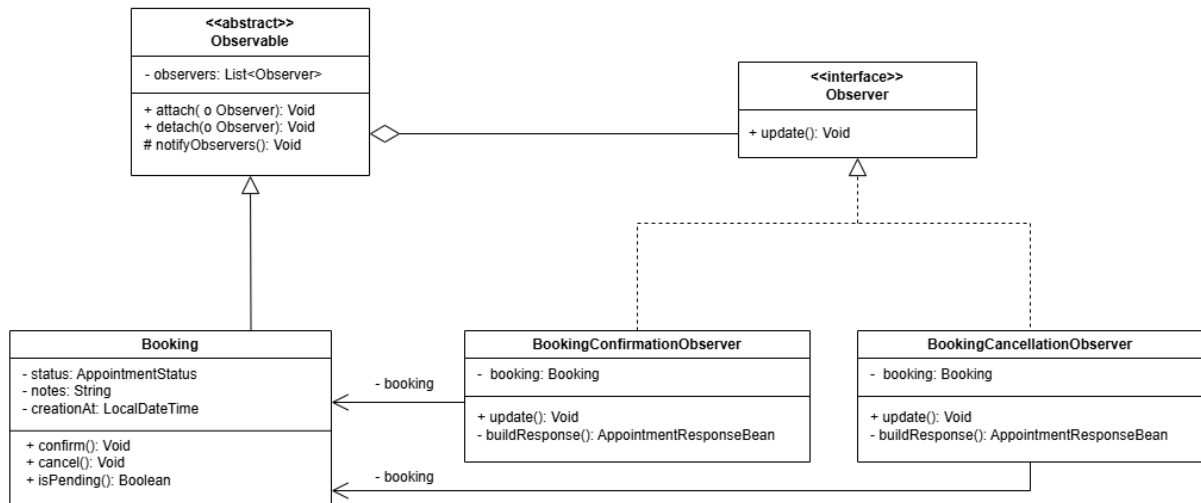


3.1.2 Design-Level Diagram

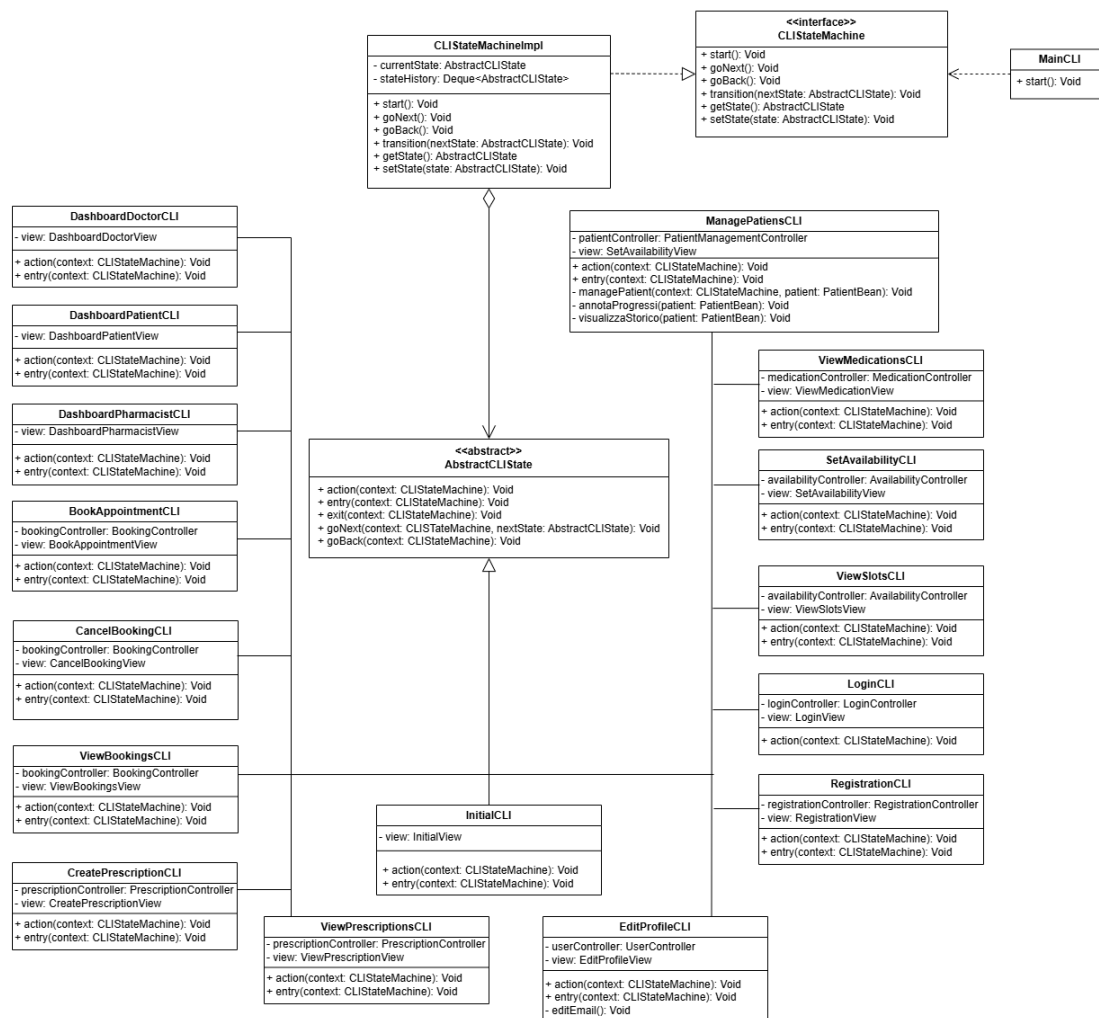
Singleton Pattern:



Observer Pattern:



State Pattern:



3.2 Design patterns

1.Singleton

I adopted the Singleton pattern for SessionManager because the application needed a single, shared point to store session data (logged-in user and their role) accessible from any controller without passing it around explicitly. In fact, it guarantees that a class has exactly one instance across the entire application. It was implemented using the Initialization-on-demand Holder idiom. It consists in a private static inner class Holder that holds the unique instance, ensuring lazy initialization and thread safety without synchronized. Furthermore, SessionManager exposes role checking methods that allow controllers to check the current user's role without accessing the User object directly and keeping authorization logic centralized.

2.Observer

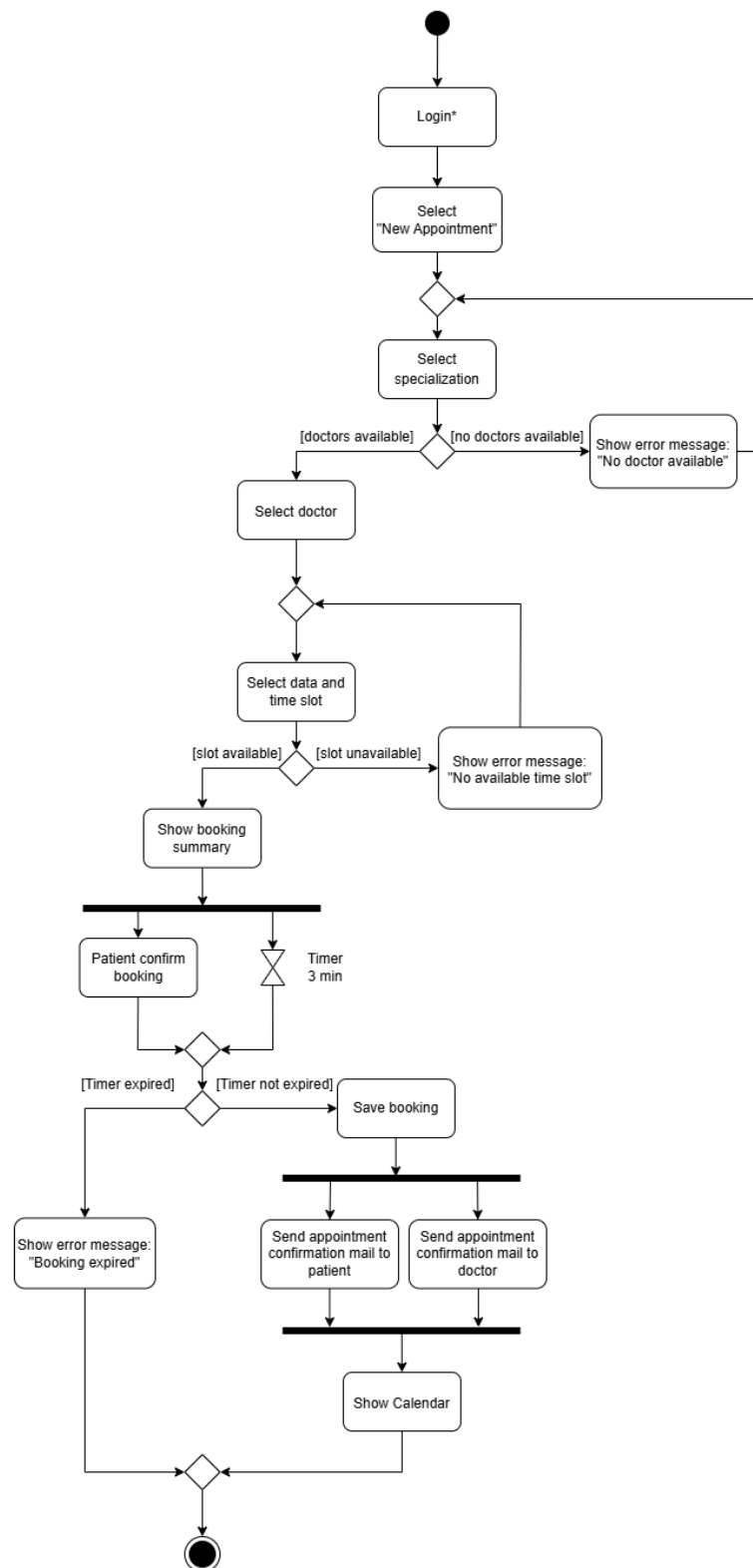
I used the Observer pattern to decouple appointment state changes from notification handling. Neither Booking nor the AppointmentController know about NotificationService. The mechanism relies on an abstract class Observable, extended by Booking, which maintains a list of Observer instances and manages their attachment, detachment, and notification via notifyObservers(). The observers are attached in AppointmentController before calling confirm() or cancel(). When a state change occurs, it triggers the appropriate email notification to both patient and doctor. Two concrete observers are defined: BookingConfirmationObserver and BookingCancellationObserver, both implementing the Observer interface. Each concrete observer holds a reference to a Booking and uses a buildResponse() method alongside update() to read the updated state and construct the response directly from the Booking instance. The design is easily extensible: new notification channels (e.g. SMS, push) or new state changes can be handled by simply adding a new observer implementing the Observer interface without modifying existing classes.

3.State

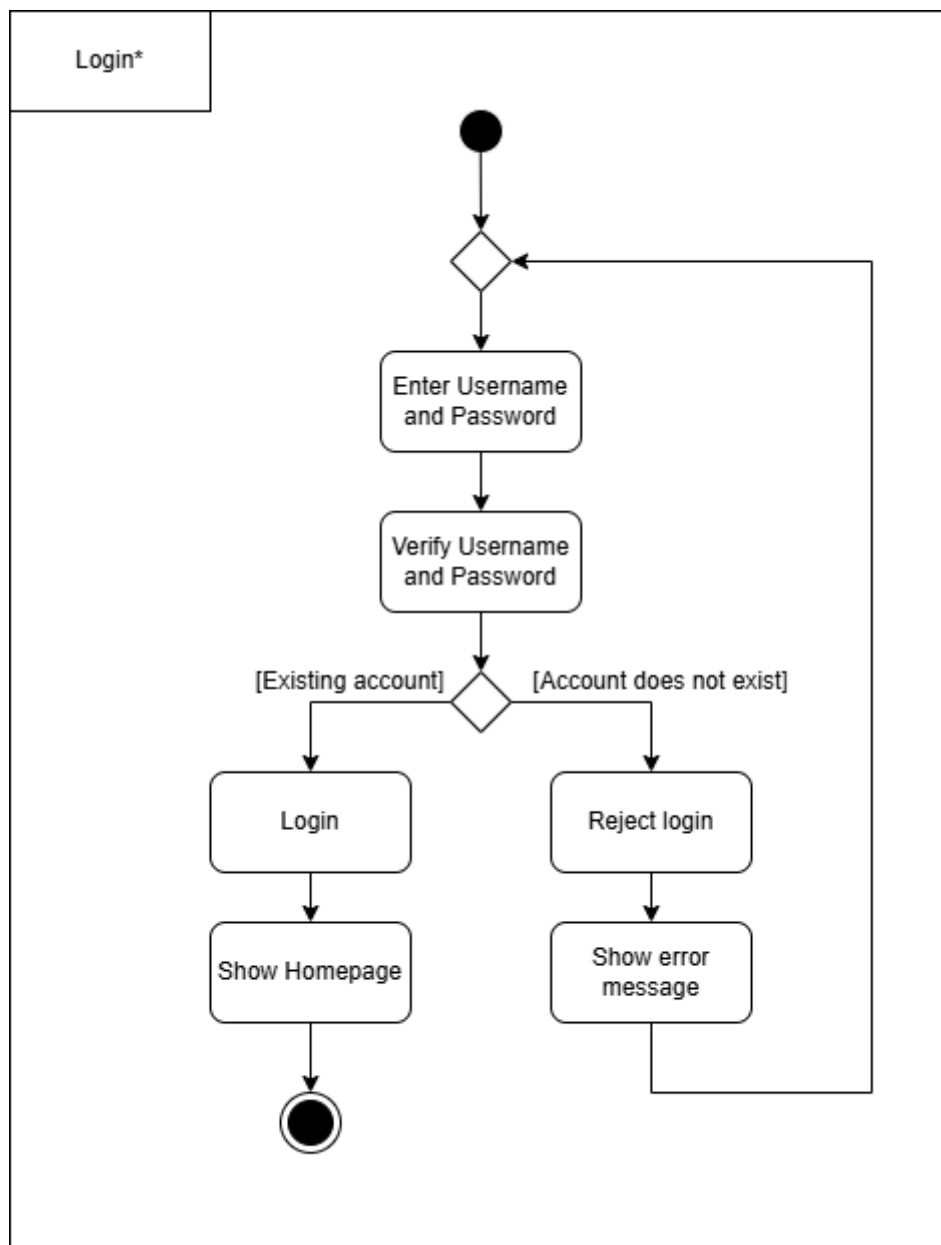
I chose the State pattern to manage navigation across the application's CLI screens, avoiding centralized control structures. MainCLI acts as the Client, simply invoking the start() method on the state machine. CLIStateMachineImpl serves as the Context implementing the CLIStateMachine interface, holding the currentState and a state history stack (stateHistory as a Deque<AbstractCLIState>) to support forward and backward navigation via methods like goNext() and goBack(). All individual screens extend the AbstractCLIState abstract class, which defines the core lifecycle methods: action(CLIStateMachine context) for handling user input and transition logic, and entry(CLIStateMachine context) for screen initialization. Each concrete state manages its own transitions via a switch-case inside action(), keeping each screen self-contained and making the system open to extension without modifying existing classes.

3.3 Activity diagram

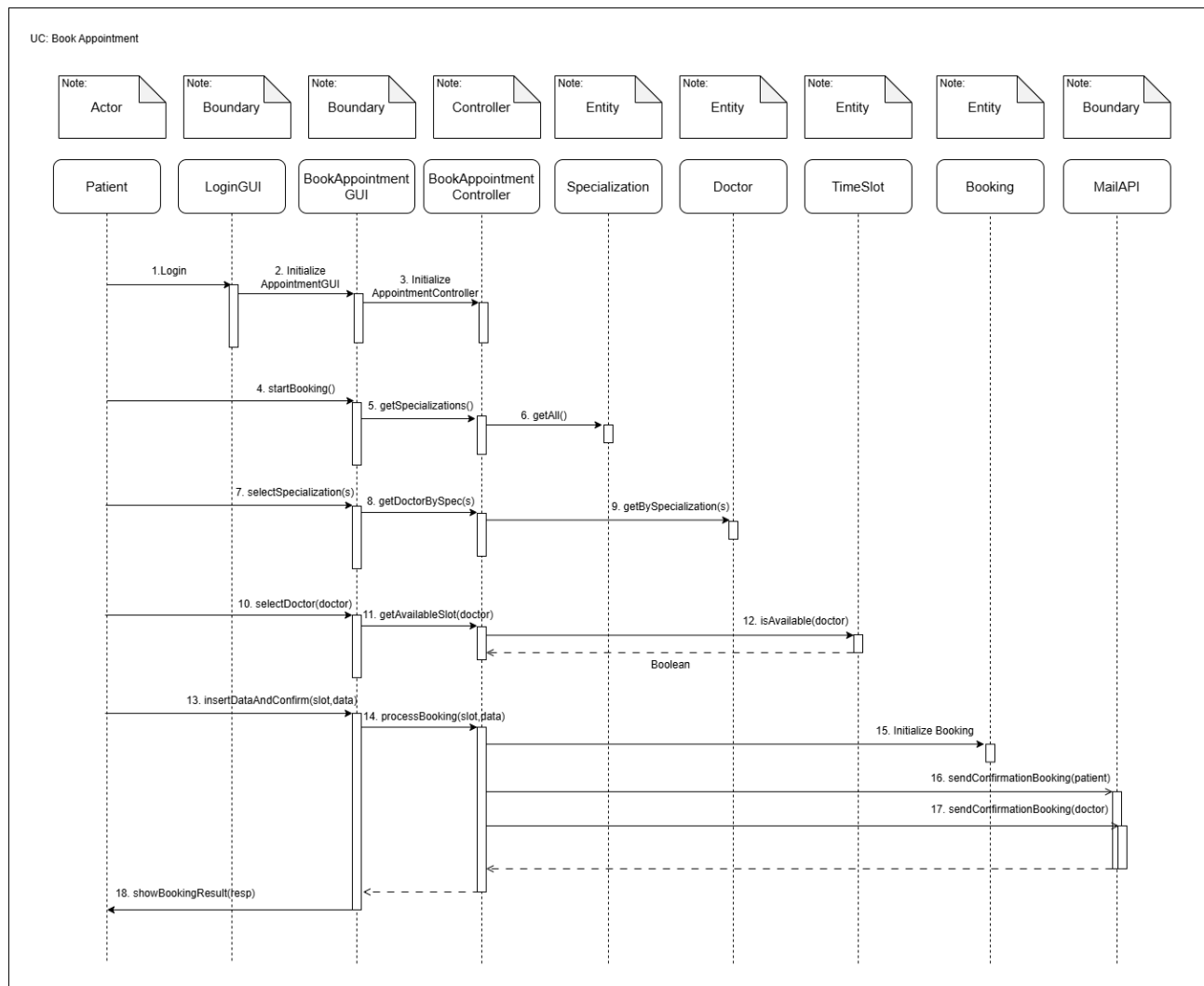
3.3.1 UC: Book Appointment



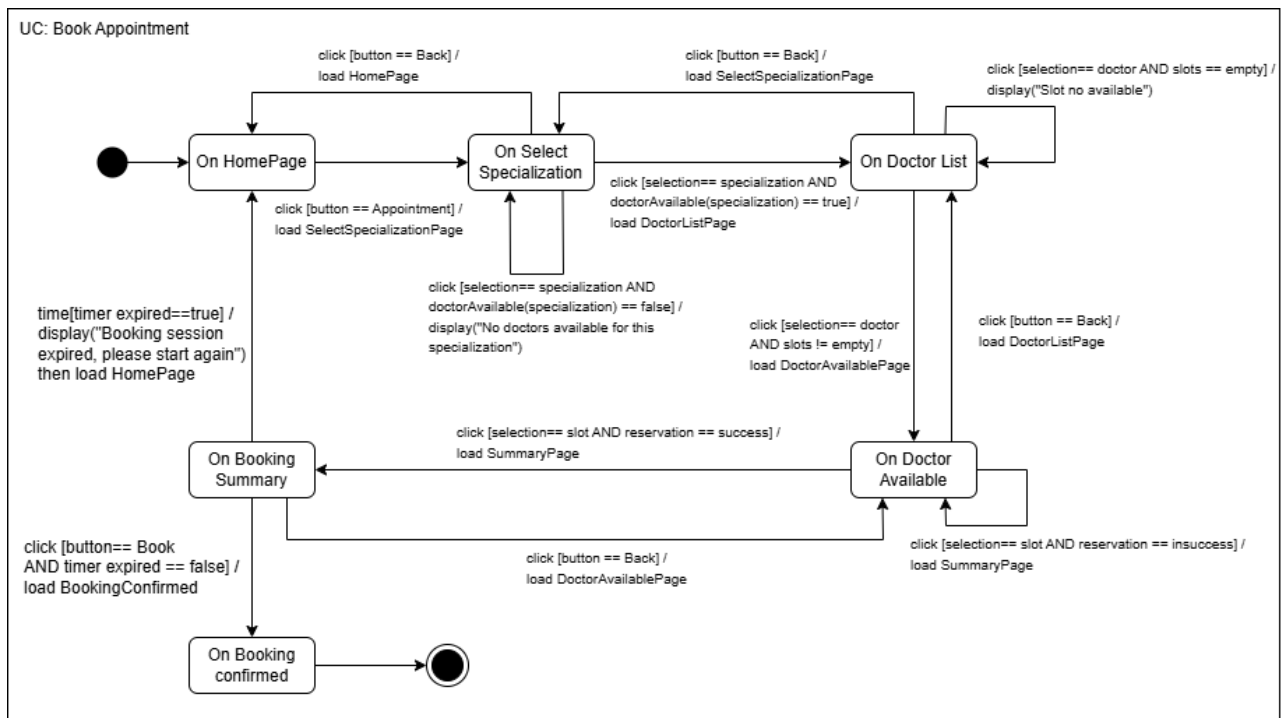
3.3.2 UC: Login



3.4 Sequence Diagram



3.5 State Diagram



4 Testing

1. Detection of overlapping availability slots

The slot availability management was tested by verifying that a doctor cannot add two overlapping time slots. The first slot is added successfully, while a second slot with an overlapping time range correctly raises an AvailabilityException.

2. Booking on an already reserved slot

The test verifies the reservation mechanism through BookingController: when Patient 1 successfully prepares a booking summary for a specific time slot, the slot gets temporarily reserved. A second attempt by Patient 2 on the same slot correctly raises a BookingException, ensuring that overlapping or double reservations on an already committed slot are properly blocked.

3. Login with invalid credentials

The authentication system was tested by attempting a login with an unregistered email address. This test ensures that the system correctly rejects invalid credentials by raising a LoginException, guaranteeing that only registered users can access the platform.

4. Rejection of duplicate registrations

The system was verified to prevent two accounts from sharing the same email address. After a first successful registration, a second attempt with the same email must raise a `RegistrationException`, ensuring the uniqueness of user identities within the platform.

5 GitHub Repository

[Evangelisti-Francesco/DoseGuard](#)

6 SonarCloud

[Overview](#)

7 Presentation Video

[DoseGuard video presentation](#)